

Sean McHugh
October 24, 2025
Data Analytics Lab 4

```
#####  
> ### Principal Component Analysis (PCA) ###  
> #####  
>  
> ## load libraries  
> library(ggplot2)  
> library(ggfortify)  
> library(GGally)  
> library(e1071)  
> library(class)  
> library(psych)  
> library(readr)  
>  
> ## set working directory so that files can be referenced without the full path  
> setwd("~/Downloads")  
>  
> ## read dataset  
> wine <- read_csv("wine.data", col_names = FALSE)  
Rows: 178 Columns: 14  
— Column specification
```

Delimiter: ","
dbl (14): X1, X2, X3, X4, X5, X6, X7, X8, X9, X10, X11, X12, X13, X14

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.
>
> ## set column names
> names(wine) <- c("Type", "Alcohol", "Malic acid", "Ash", "Alcalinity of ash", "Magnesium", "Total phenols", "Flavanoids", "Nonflavanoid Phenols", "Proanthocyanins", "Color Intensity", "Hue", "Od280/od315 of diluted wines", "Proline")
>
> ## inspect data frame
> head(wine)
A tibble: 6 × 14
 Type Alcohol `Malic acid` Ash `Alcalinity of ash` Magnesium `Total phenols` Flavanoids
 `Nonflavanoid Phenols` Proanthocyanins `Color Intensity` Hue

	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	1	14.2	1.71	2.43	15.6	127	2.8	3.06	0.28
		2.29	5.64	1.04					
2	1	13.2	1.78	2.14	11.2	100	2.65	2.76	0.26
		1.28	4.38	1.05					
3	1	13.2	2.36	2.67	18.6	101	2.8	3.24	0.3
		2.81	5.68	1.03					
4	1	14.4	1.95	2.5	16.8	113	3.85	3.49	0.24
		2.18	7.8	0.86					
5	1	13.2	2.59	2.87	21	118	2.8	2.69	0.39
		1.82	4.32	1.04					
6	1	14.2	1.76	2.45	15.2	112	3.27	3.39	0.34
		1.97	6.75	1.05					

i 2 more variables: `Od280/od315 of diluted wines` <dbl>, Proline <dbl>

>

> ## change the data type of the "Type" column from character to factor

> #####

> # Factors look like regular strings (characters) but with factors R knows

> # that the column is a categorical variable with finite possible values

> # e.g. "Type" in the Wine dataset can only be 1, 2, or 3

> #####

>

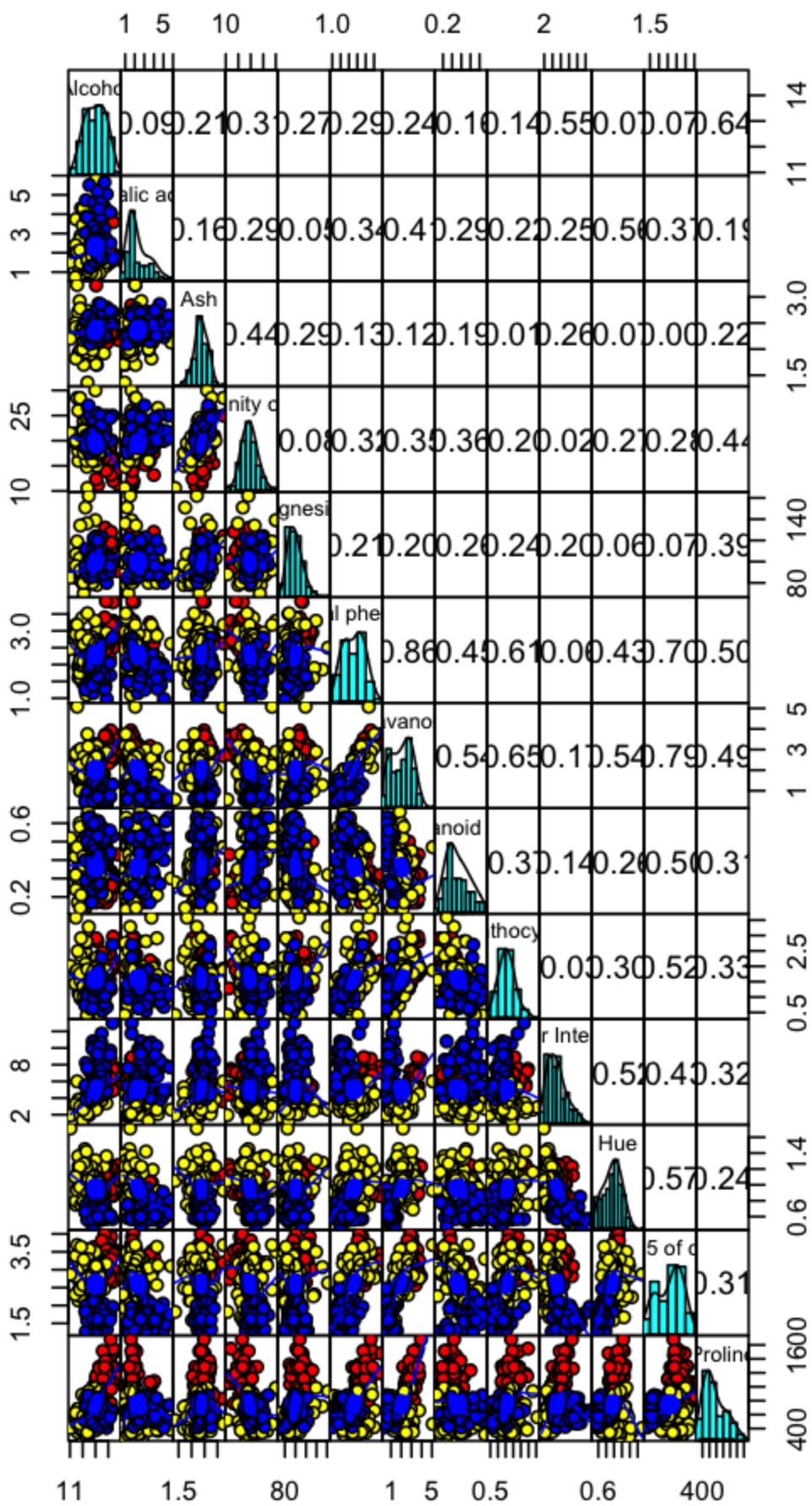
> wine\$Type <- as.factor(wine\$Type)

>

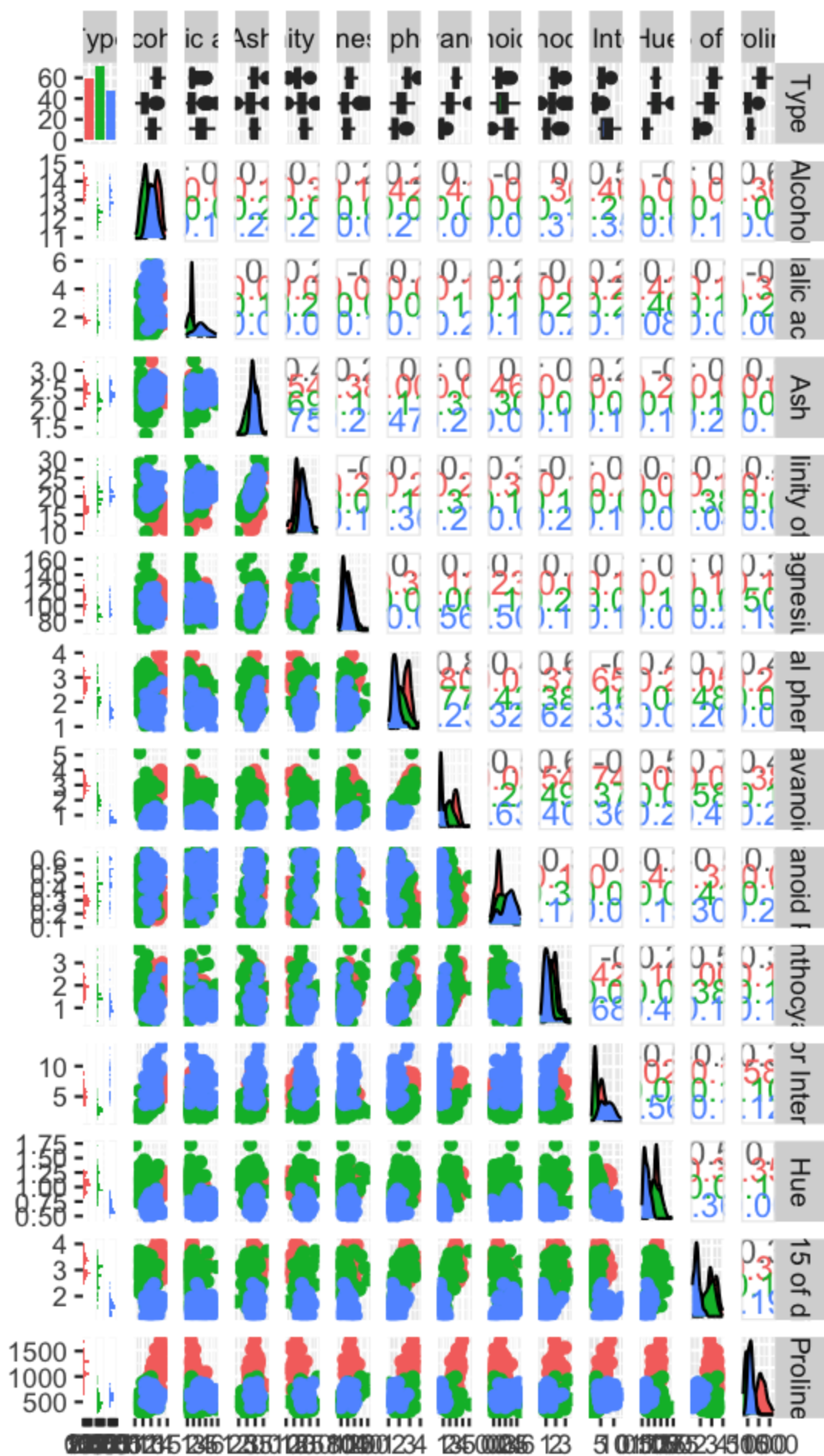
>

> ## visualize variables

> pairs.panels(wine[,-1],gap = 0,bg = c("red", "yellow", "blue")[wine\$Type],pch=21)



```
>  
> ggpairs(wine, ggplot2::aes(colour = Type))
```



```

>

> ###
> # creating another dataframe from wine dataset that contains the columns from 2 to 14
> X <- wine[,2:14]
> Y <- wine[,1]
>
> ## Compute the PCs and plot the dataset using the 1st and 2nd PCs.
>
> Xmat <- as.matrix(X)
>
> Xc <- scale(Xmat, center = T, scale = F)
>
> principal_components <- princomp(Xc)
>
> summary(principal_components)
Importance of components:
              Comp.1   Comp.2   Comp.3   Comp.4   Comp.5   Comp.6
Comp.7   Comp.8   Comp.9   Comp.10   Comp.11
Standard deviation  314.0771817 13.098319252 3.063510e+00 2.227810e+00 1.105415e+00
9.145156e-01 5.266937e-01 3.879830e-01 3.338668e-01 2.670202e-01 1.933000e-01
Proportion of Variance  0.9980912 0.001735916 9.495896e-05 5.021736e-05 1.236368e-05
8.462130e-06 2.806815e-06 1.523081e-06 1.127830e-06 7.214158e-07 3.780603e-07
Cumulative Proportion  0.9980912 0.999827146 9.999221e-01 9.999723e-01 9.999847e-01
9.999931e-01 9.999960e-01 9.999975e-01 9.999986e-01 9.999993e-01 9.999997e-01
              Comp.12   Comp.13
Standard deviation  1.447549e-01 9.031952e-02
Proportion of Variance 2.120138e-07 8.253928e-08
Cumulative Proportion 9.999999e-01 1.000000e+00
>
> principal_components$loadings

```

```

Loadings:
              Comp.1 Comp.2 Comp.3 Comp.4 Comp.5 Comp.6 Comp.7 Comp.8 Comp.9
Comp.10 Comp.11 Comp.12 Comp.13
Alcohol              0.141      0.194 0.923 0.285
Malic acid           0.122 0.160 -0.613 0.742 -0.150
Ash                  -0.149      -0.954 0.132 -0.174
Alcalinity of ash    0.939 -0.331
Magnesium            0.999
Total phenols        0.315 0.279   -0.177 0.256 -0.847
Flavanoids          -0.169 0.525 0.434   -0.248 0.378 0.520 0.133
Nonflavanoid Phenols      -0.199 -0.148 0.966
Proanthocyanins       0.251 0.242 -0.310 0.870      -0.136

```

Color Intensity	0.291	0.879	0.332	-0.113							
Hue								-0.976	-0.167		
Od280/od315 of diluted wines				-0.178	0.261	0.289	0.102	-0.187	-0.874		
Proline	1.000										

	Comp.1	Comp.2	Comp.3	Comp.4	Comp.5	Comp.6	Comp.7	Comp.8	Comp.9	Comp.10	Comp.11	Comp.12	Comp.13
SS loadings	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
Proportion Var	0.077	0.077	0.077	0.077	0.077	0.077	0.077	0.077	0.077	0.077	0.077	0.077	0.077
Cumulative Var	0.077	0.154	0.231	0.308	0.385	0.462	0.538	0.615	0.692	0.769	0.846	0.923	1.000

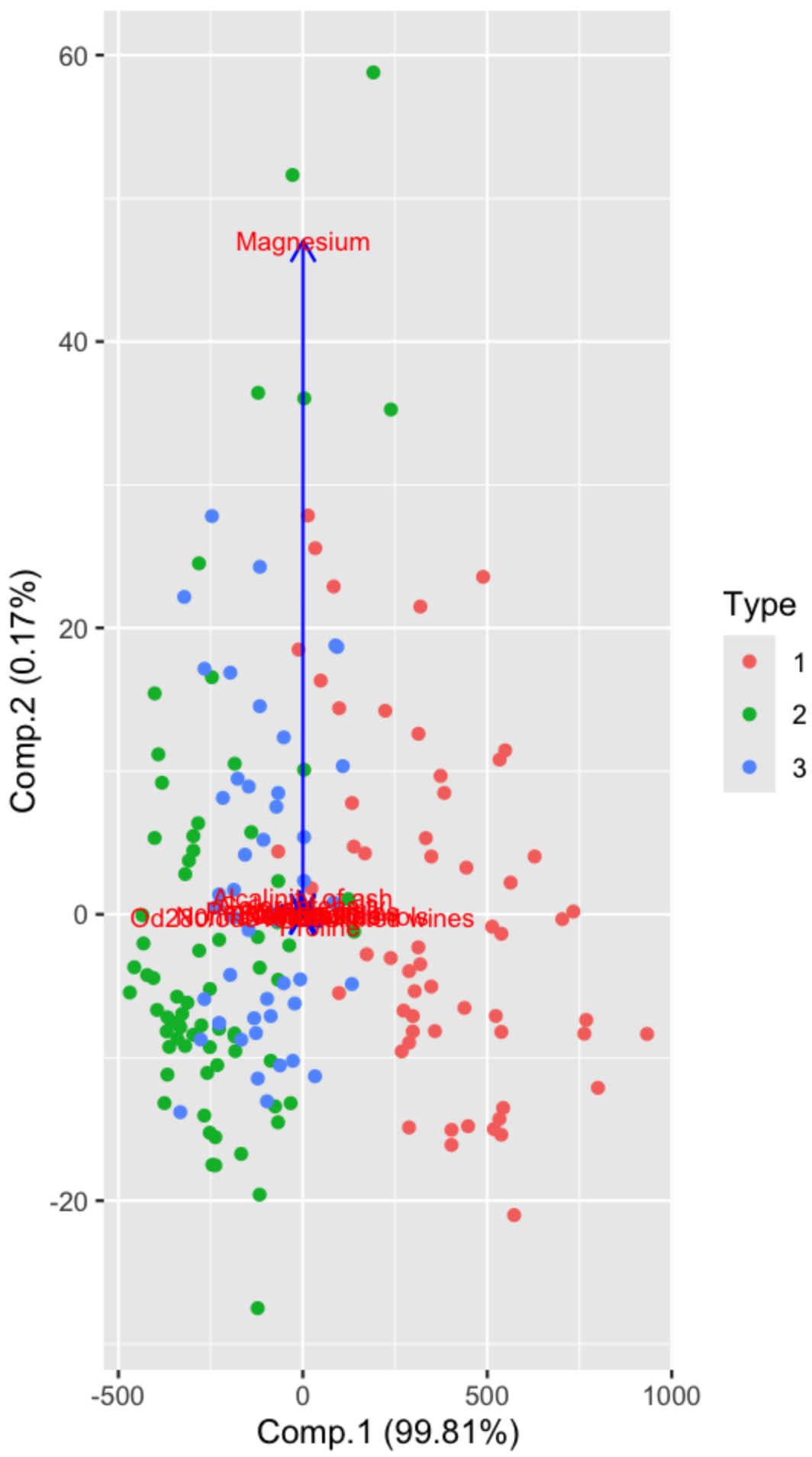
>

> # using autoplot() function to plot the components

> autoplot(principal_components, data = wine, colour = 'Type',

+ loadings = TRUE, loadings.colour = 'blue',

+ loadings.label = TRUE, loadings.label.size = 3, scale = 0)




```

>
> # Following the same steps except with scaled PCA
> Xs <- scale(Xmat, center = T, scale = T)
>
> principal_components2 <- princomp(Xs)
>
> summary(principal_components2)
Importance of components:
      Comp.1  Comp.2  Comp.3  Comp.4  Comp.5  Comp.6  Comp.7
Comp.8  Comp.9  Comp.10  Comp.11  Comp.12  Comp.13
Standard deviation  2.1631951 1.5757366 1.1991447 0.9559347 0.92110518 0.79878171
0.74022473 0.58867607 0.53596364 0.49949266 0.47383559 0.40966094 0.320619963
Proportion of Variance 0.3619885 0.1920749 0.1112363 0.0706903 0.06563294 0.04935823
0.04238679 0.02680749 0.02222153 0.01930019 0.01736836 0.01298233 0.007952149
Cumulative Proportion 0.3619885 0.5540634 0.6652997 0.7359900 0.80162293 0.85098116
0.89336795 0.92017544 0.94239698 0.96169717 0.97906553 0.99204785 1.000000000
>
> principal_components2$loadings

```

Loadings:

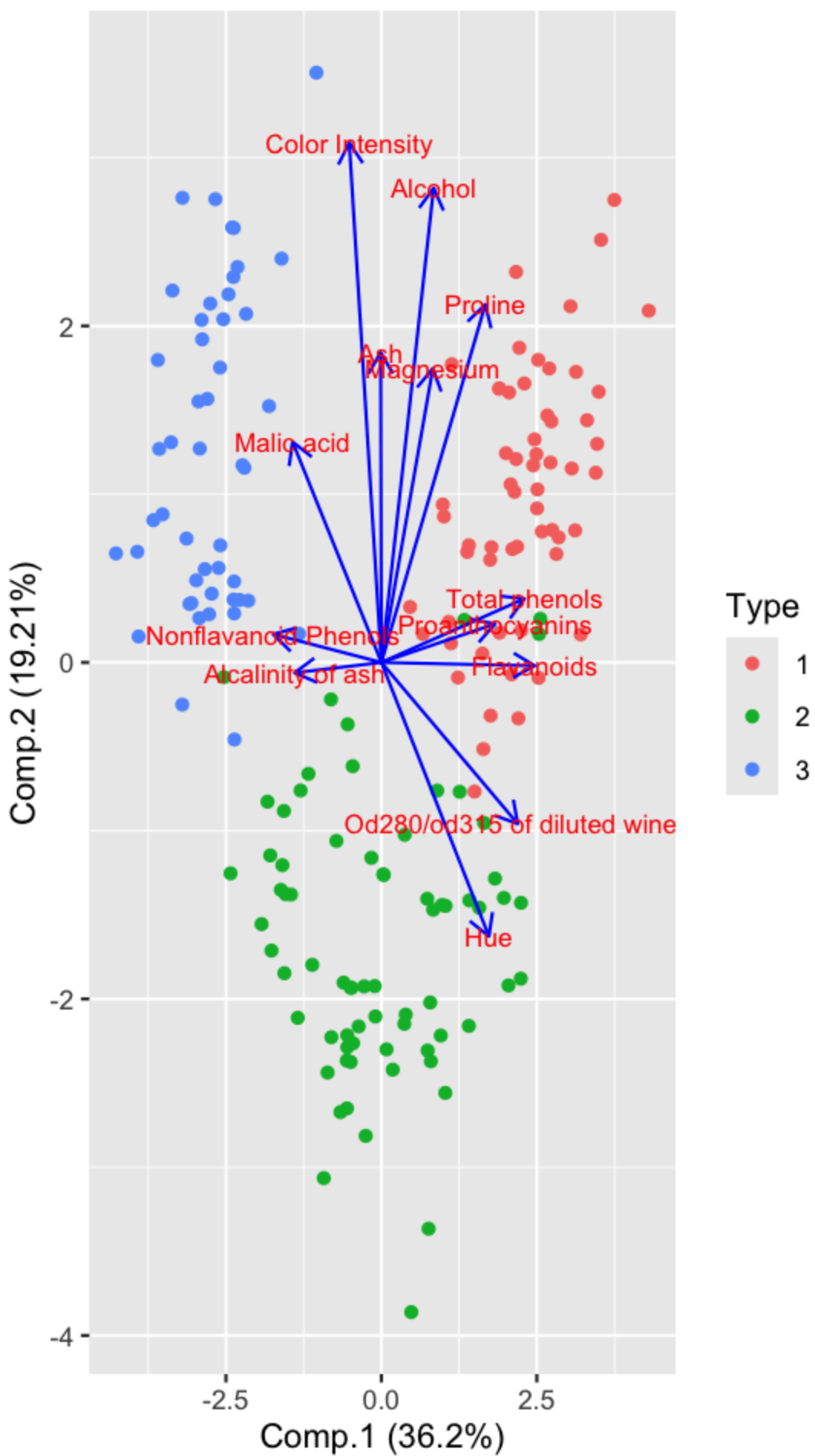
	Comp.1	Comp.2	Comp.3	Comp.4	Comp.5	Comp.6	Comp.7	Comp.8	Comp.9
Alcohol	0.144	0.484	0.207	0.266	0.214	0.396	0.509	0.212	0.226
Malic acid	-0.245	0.225	-0.537	0.537	-0.421		-0.309		-0.122
Ash	0.316	-0.626	0.214	0.143	0.154	0.149	-0.170	-0.308	0.499
Alcalinity of ash	-0.239	-0.612		-0.101	0.287	0.428	0.200		-0.479
Magnesium	0.142	0.300	-0.131	0.352	-0.727	-0.323	-0.156	0.271	
Total phenols	0.395	-0.146	-0.198	0.149		-0.406	0.286	-0.320	-0.304
Flavanoids	0.423	-0.151	-0.152	0.109		-0.187		-0.163	
Nonflavanoid Phenols	-0.299	-0.170	0.203	0.501	-0.259	-0.595	-0.233	0.196	0.216
Proanthocyanins	0.313	-0.149	-0.399	-0.137	-0.534	-0.372	0.368	-0.209	0.134
Color Intensity	0.530	0.137		-0.419	0.228		-0.291		-0.604
Hue	0.297	-0.279	0.428	0.174	0.106	-0.232	0.437	-0.522	
Od280/od315 of diluted wines	0.376	-0.164	-0.166	-0.184	0.101	0.266		0.137	0.524
Proline	0.287	0.365	0.127	0.232	0.158	0.120	0.120	-0.576	0.162

	Comp.1	Comp.2	Comp.3	Comp.4	Comp.5	Comp.6	Comp.7	Comp.8	Comp.9	Comp.10
Comp.11	Comp.12	Comp.13								
SS loadings	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000
1.000	1.000									
Proportion Var	0.077	0.077	0.077	0.077	0.077	0.077	0.077	0.077	0.077	0.077
0.077	0.077									
Cumulative Var	0.077	0.154	0.231	0.308	0.385	0.462	0.538	0.615	0.692	0.769
0.923	1.000									

```

>
> library(caret)
>
> # using autoplot() function to plot the components
> autoplot(principal_components2, data = wine, colour = 'Type',
+   loadings = TRUE, loadings.colour = 'blue',
+   loadings.label = TRUE, loadings.label.size = 3, scale = 0)

```



```

> ## Identify the variables that contribute the most to the 1st PC and 2nd PCs.
> # Color intensity, alcohol, proline, total phenols, flavanoids, Od280/od315 of diluted wines, and
hue contribute the most
>
> ## Train a classifier model (e.g. kNN) to predict the wine type using all the variables in the
original dataset.
> split.rat <- 0.7
> train.indexes <- sample(150,split.rat*150)
>
> train <- wine[train.indexes,]
> test <- wine[-train.indexes,]
>
> mod.knn <- train(Type~., data=train, method="knn")
>
> ### 1 round cross-validation
> knn.train.true <- train[,1]
> knn.test.true <- test[,1]
>
> knn.train.predicted <- predict(mod.knn,train[,-1])
> knn.test.predicted <- predict(mod.knn,test[,-1])
>
> train.cm = as.matrix(table(Actual = knn.train.true$Type, Predicted = knn.train.predicted))
> train.cm
      Predicted
Actual 1 2 3
      1 39 2 0
      2 5 45 3
      3 3 8 0
> train.accuracy <- sum(diag(train.cm))/nrow(train)
> train.accuracy
[1] 0.8
>
> test.cm = as.matrix(table(Actual = knn.test.true$Type, Predicted = knn.test.predicted))
> test.cm
      Predicted
Actual 1 2 3
      1 17 1 0
      2 0 17 1
      3 6 30 1
> test.accuracy <- sum(diag(test.cm))/nrow(test)
> test.accuracy
[1] 0.4794521
>

```

```

> ## Train a classifier model to predict the wine type using the data projected onto the first 2 PCs
(scores in the princomp function's return object)
> pc_scores <- as.data.frame(principal_components2$scores[, 1:2])
> colnames(pc_scores) <- c("PC1", "PC2")
> pc_data <- data.frame(Type = as.factor(wine$Type), pc_scores)
>
> train.pc <- pc_data[train.indexes, ]
> test.pc <- pc_data[-train.indexes, ]
>
> mod.knn <- train(Type ~ PC1 + PC2, data = train.pc, method = "knn")
>
> knn.train.true <- train.pc[, 1]; knn.test.true <- test.pc[, 1]
> knn.train.predicted <- predict(mod.knn, train.pc[, -1])
> knn.test.predicted <- predict(mod.knn, test.pc[, -1])
>
> train.cm <- as.matrix(table(Actual = knn.train.true, Predicted = knn.train.predicted))
> test.cm <- as.matrix(table(Actual = knn.test.true, Predicted = knn.test.predicted))
>
> ## Compare the 2 classification models using contingency tables and precision/recall/F1
metrics.
> train.cm <- as.matrix(table(Actual = train.pc$Type, Predicted = knn.train.predicted))
> train.cm
      Predicted
Actual 1 2 3
      1 40 1 0
      2 3 49 1
      3 0 0 11
> train.accuracy <- sum(diag(train.cm)) / nrow(train.pc)
> train.accuracy
[1] 0.952381
>
> test.cm <- as.matrix(table(Actual = test.pc$Type, Predicted = knn.test.predicted))
> test.cm
      Predicted
Actual 1 2 3
      1 18 0 0
      2 0 18 0
      3 0 1 36
> test.accuracy <- sum(diag(test.cm)) / nrow(test.pc)
> test.accuracy
[1] 0.9863014
>
> knn.cm.full <- confusionMatrix(knn.test.predicted, test.pc$Type, mode = "prec_recall")
> knn.byclass <- as.data.frame(knn.cm.full$byClass)

```

```
>
> knn.precision_macro <- mean(knn.byclass$Precision)
> knn.recall_macro    <- mean(knn.byclass$Recall)
> knn.f1_macro        <- mean(knn.byclass$F1)
>
> knn.precision_macro
[1] 0.9824561
> knn.recall_macro
[1] 0.990991
> knn.f1_macro
[1] 0.9864248
```